

FIT – HPUni BECOME A BETTER DEVELOPER

=====

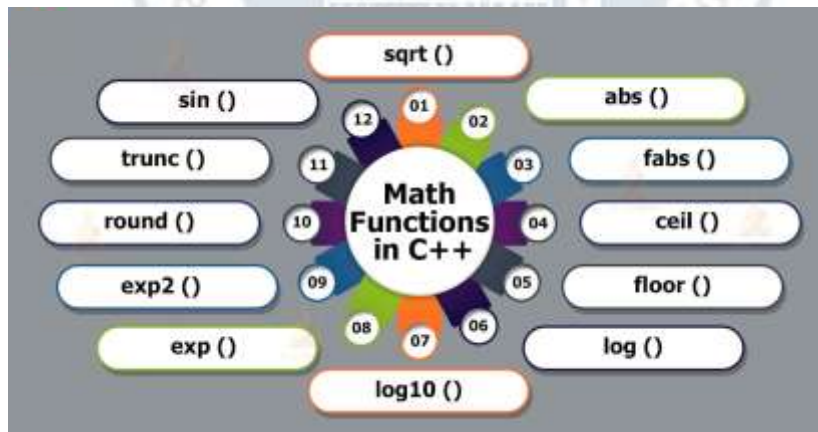
BỘ BÀI TẬP THỰC HÀNH HÀM TRONG C++

30 bài: hàm void - tham số giá trị - tham số tham chiếu/con trỏ

Mỗi bài gồm: Nêu bài toán - Input - Output - Ví dụ - Thuật toán - Code minh họa

10 bài mỗi nhóm | Chuẩn C++11 | 30 chương trình đã kiểm tra biên dịch và dữ liệu ví dụ

Mục tiêu: rèn luyện việc tách chương trình thành hàm, phân biệt hàm không trả về giá trị với hàm trả kết quả, hiểu cơ chế truyền tham số theo giá trị và cách dùng tham chiếu/con trỏ khi cần thay đổi đối tượng của hàm gọi hoặc trả nhiều kết quả.



HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

Tài liệu gồm đúng 30 bài, chia thành ba phần bằng nhau: 10 bài về hàm void, 10 bài về hàm sử dụng tham số giá trị (tham số trị) và 10 bài về tham số tham chiếu hoặc tham số con trỏ (tham số biến).

Mỗi bài được trình bày thống nhất theo 6 mục: (1) Nêu bài toán, (2) Input, (3) Output, (4) Ví dụ, (5) Thuật toán giải và (6) Code minh họa.

Các chương trình dùng cú pháp tương thích C++11 và đã được biên dịch bằng g++ với các tùy chọn cảnh báo, sau đó chạy đối chiếu với dữ liệu ví dụ.

Cơ sở tham khảo từ tài liệu đính kèm

- Bjarne Stroustrup, The C++ Programming Language, Fourth Edition, Chương 12: khi gọi hàm, các đối số thực được dùng để khởi tạo các tham số hình thức; nếu tham số không phải là reference thì một bản sao của đối số được truyền vào hàm.
- Tham số truyền theo giá trị phù hợp cho các đối tượng nhỏ; việc thay đổi tham số cục bộ không làm thay đổi biến tương ứng ở hàm gọi.
- Một tham chiếu không const thường thể hiện ý định cho phép hàm sửa đổi đối tượng được truyền vào; const reference phù hợp khi chỉ cần đọc đối tượng lớn mà không muốn sao chép.
- Con trỏ có thể dùng làm tham số khi cần biểu diễn địa chỉ hoặc khả năng “không có đối tượng” bằng nullptr; phải kiểm tra tính hợp lệ trước khi giải tham chiếu.
- Đối với mảng built-in truyền vào hàm, tham số mảng được chuyển thành con trỏ tới phần tử đầu; kích thước mảng không tự đi kèm nên cần truyền n riêng.

Phân bố 30 bài tập

Phần	Bài	Số bài	Trọng tâm
I	1-10	10	Hàm void: thực hiện tác vụ/in kết quả, không trả về giá trị
II	11-20	10	Tham số giá trị: hàm nhận bản sao đối số và trả về kết quả
III	21-30	10	Tham số tham chiếu hoặc con trỏ: thay đổi biến gọi/trả nhiều kết quả

Danh mục bài tập

Bài	Tên bài	Nhóm	Kỹ thuật chính
1	In tổng hai số bằng hàm void	Hàm void	void + tham số đầu vào
2	In chu vi và diện tích hình chữ nhật	Hàm void	void + nhiều kết quả in trực tiếp
3	Phân loại một số nguyên	Hàm void	void + if/else
4	In bảng cửu chương của một số	Hàm void	void + vòng lặp for
5	In các số chẵn từ 1 đến n	Hàm void	void + duyệt miền giá trị
6	In các ước dương của n	Hàm void	void + phép chia dư
7	In các chữ số theo thứ tự đảo ngược	Hàm void	void + xử lý chữ số
8	In các số nguyên tố không vượt quá n	Hàm void	void + kiểm tra nguyên tố
9	Vẽ hình chữ nhật bằng ký tự *	Hàm void	void + vòng lặp lồng nhau
10	In n số Fibonacci đầu tiên	Hàm void	void + cập nhật trạng thái cục bộ
11	Tính trị tuyệt đối bằng tham số trị	Tham số giá trị	int absValue(int x)
12	Tìm số lớn nhất trong ba số	Tham số giá trị	int max3(int a, int b, int c)
13	Tính giai thừa n!	Tham số giá trị	long long factorial(int n)
14	Kiểm tra số nguyên tố	Tham số giá trị	bool isPrime(int n)
15	Tìm ước chung lớn nhất	Tham số giá trị	int gcdValue(int a, int b)
16	Tính bội chung nhỏ nhất	Tham số giá trị	return value + helper function

17	Tính lũy thừa nguyên a^n	Tham số giá trị	long long powerValue(long long a, int n)
18	Đếm số chữ số của số nguyên không âm	Tham số giá trị	int countDigits(int n)
19	Tính tổng các chữ số	Tham số giá trị	int sumDigits(int n)
20	Đảo ngược một số nguyên không âm	Tham số giá trị	long long reverseNumber(int n)
21	Hoán đổi hai số bằng tham chiếu	Tham số tham chiếu/con trỏ	void swapRef(int& a, int& b)
22	Sắp xếp ba số tăng dần bằng tham chiếu	Tham số tham chiếu/con trỏ	nhiều int&
23	Trả đồng thời giá trị nhỏ nhất và lớn nhất	Tham số tham chiếu/con trỏ	output parameters bằng reference
24	Trả thương và số dư bằng tham chiếu	Tham số tham chiếu/con trỏ	hai output references
25	Đổi tổng số giây thành giờ, phút, giây	Tham số tham chiếu/con trỏ	ba output references
26	Nhân đôi giá trị bằng tham số con trỏ	Tham số tham chiếu/con trỏ	void doubleValue(int* p)
27	Hoán đổi hai số bằng tham số con trỏ	Tham số tham chiếu/con trỏ	void swapPtr(int* a, int* b)
28	Tăng mọi phần tử mảng thêm k qua con trỏ	Tham số tham chiếu/con trỏ	int* a + kích thước n
29	Tìm min và max của mảng qua con trỏ kết quả	Tham số tham chiếu/con trỏ	const int* + int* output
30	Tính chu vi và diện tích hình tròn qua con trỏ kết quả	Tham số tham chiếu/con trỏ	double* output parameters



PHẦN I. HÀM VOID

Tập trung vào kiểu trả về void: hàm thực hiện một công việc như in kết quả hoặc biến đổi luồng xuất, không dùng return để trả một giá trị cho biểu thức gọi.

Bài 1. In tổng hai số bằng hàm void

1. Nêu bài toán

Nhập hai số nguyên a và b. Viết hàm void nhận hai số và in tổng của chúng.

2. Input

Hai số nguyên a, b.

3. Output

Tổng a + b.

4. Ví dụ

Input	Output
7 12	19

5. Thuật toán giải

Bước 1: Đọc a và b trong hàm main.

Bước 2: Xây dựng hàm void printSum(int a, int b).

Bước 3: Trong hàm, tính a + b và in trực tiếp kết quả.

Bước 4: Gọi printSum(a, b) từ main.

6. Code minh họa

```
#include <iostream>
using namespace std;

void printSum(int a, int b) {
    cout << a + b;
}

int main() {
    int a, b;
    cin >> a >> b;
    printSum(a, b);
    return 0;
}
```

Bài 2. In chu vi và diện tích hình chữ nhật

1. Nêu bài toán

Nhập chiều dài và chiều rộng của hình chữ nhật. Viết hàm void tính và in chu vi, diện tích.

2. Input

Hai số thực dương a, b.

3. Output

Dòng 1: chu vi. Dòng 2: diện tích. In với 2 chữ số thập phân.

4. Ví dụ

Input	Output
-------	--------

53

16.00

15.00

5. Thuật toán giải

Bước 1: Đọc chiều dài a và chiều rộng b.

Bước 2: Hàm printRectangleInfo(a, b) tính $p = 2 \cdot (a+b)$ và $s = a \cdot b$.

Bước 3: Định dạng fixed, setprecision(2).

Bước 4: In p và s trên hai dòng.

6. Code minh họa

```
#include <iostream>
#include <iomanip>
using namespace std;

void printRectangleInfo(double a, double b) {
    double p = 2 * (a + b);
    double s = a * b;
    cout << fixed << setprecision(2);
    cout << p << '\n' << s;
}

int main() {
    double a, b;
    cin >> a >> b;
    printRectangleInfo(a, b);
    return 0;
}
```

Bài 3. Phân loại một số nguyên**1. Nêu bài toán**

Nhập số nguyên x. Viết hàm void in POSITIVE nếu $x > 0$, NEGATIVE nếu $x < 0$ và ZERO nếu $x = 0$.

2. Input

Một số nguyên x.

3. Output

Một trong ba chuỗi POSITIVE, NEGATIVE hoặc ZERO.

4. Ví dụ

Input	Output
-25	NEGATIVE

5. Thuật toán giải

Bước 1: Đọc x.

Bước 2: Hàm printSign(x) kiểm tra x theo ba trường hợp.

Bước 3: In chuỗi tương ứng; hàm không trả về giá trị.

6. Code minh họa

```
#include <iostream>
using namespace std;

void printSign(int x) {
    if (x > 0) cout << "POSITIVE";
}
```

```

    else if (x < 0) cout << "NEGATIVE";
    else cout << "ZERO";
}

int main() {
    int x;
    cin >> x;
    printSign(x);
    return 0;
}

```

Bài 4. In bảng cửu chương của một số

1. Nêu bài toán

Nhập số nguyên n. Viết hàm void in bảng nhân của n từ 1 đến 10 theo dạng n x i = kết quả.

2. Input

Một số nguyên n.

3. Output

10 dòng biểu diễn bảng nhân từ n x 1 đến n x 10.

4. Ví dụ

Input	Output
3	3 x 1 = 3 3 x 2 = 6 3 x 3 = 9 3 x 4 = 12 3 x 5 = 15 3 x 6 = 18 3 x 7 = 21 3 x 8 = 24 3 x 9 = 27 3 x 10 = 30

5. Thuật toán giải

Bước 1: Đọc n.

Bước 2: Trong printTable(n), lặp i từ 1 đến 10.

Bước 3: Mỗi vòng lặp in n, i và n*i theo đúng định dạng.

6. Code minh họa

```

#include <iostream>
using namespace std;

void printTable(int n) {
    for (int i = 1; i <= 10; ++i) {
        cout << n << " x " << i << " = " << n * i;
        if (i < 10) cout << '\n';
    }
}

int main() {
    int n;
    cin >> n;
    printTable(n);
    return 0;
}

```

}

Bài 5. In các số chẵn từ 1 đến n**1. Nêu bài toán**

Nhập n ($n \geq 2$). Viết hàm void in tất cả số chẵn từ 1 đến n, cách nhau bởi một khoảng trắng.

2. Input

Một số nguyên n, $2 \leq n \leq 100000$.

3. Output

Dãy các số chẵn không vượt quá n.

4. Ví dụ

Input	Output
10	2 4 6 8 10

5. Thuật toán giải

Bước 1: Đọc n.

Bước 2: Hàm printEven(n) bắt đầu từ 2 và tăng 2 đơn vị.

Bước 3: In từng số; chỉ in khoảng trắng trước các phần tử sau phần tử đầu.

6. Code minh họa

```
#include <iostream>
using namespace std;

void printEven(int n) {
    bool first = true;
    for (int i = 2; i <= n; i += 2) {
        if (!first) cout << ' ';
        cout << i;
        first = false;
    }
}

int main() {
    int n;
    cin >> n;
    printEven(n);
    return 0;
}
```

Bài 6. In các ước dương của n**1. Nêu bài toán**

Nhập số nguyên dương n. Viết hàm void in các ước dương của n theo thứ tự tăng dần.

2. Input

Một số nguyên n, $1 \leq n \leq 100000$.

3. Output

Các ước dương của n, cách nhau bởi một khoảng trắng.

4. Ví dụ

Input	Output
-------	--------

5. Thuật toán giải

Bước 1: Đọc n.

Bước 2: Lặp i từ 1 đến n trong hàm printDivisors.

Bước 3: Nếu $n \% i == 0$ thì i là một ước và được in ra.

Bước 4: Kiểm soát khoảng trắng để không có khoảng trắng thừa ở đầu.

6. Code minh họa

```
#include <iostream>
using namespace std;

void printDivisors(int n) {
    bool first = true;
    for (int i = 1; i <= n; ++i) {
        if (n % i == 0) {
            if (!first) cout << ' ';
            cout << i;
            first = false;
        }
    }
}

int main() {
    int n;
    cin >> n;
    printDivisors(n);
    return 0;
}
```

Bài 7. In các chữ số theo thứ tự đảo ngược**1. Nêu bài toán**

Nhập số nguyên không âm n. Viết hàm void in các chữ số của n theo thứ tự từ phải sang trái, cách nhau bởi một khoảng trắng.

2. Input

Số nguyên n, $0 \leq n \leq 2 \cdot 10^9$.

3. Output

Các chữ số của n theo thứ tự đảo ngược.

4. Ví dụ

Input	Output
2048	8 4 0 2

5. Thuật toán giải

Bước 1: Nếu $n = 0$ thì in 0.

Bước 2: Ngược lại, lặp khi $n > 0$.

Bước 3: Lấy chữ số cuối bằng $n \% 10$, in ra rồi cập nhật $n = n / 10$.

Bước 4: Dùng biến first để định dạng khoảng trắng.

6. Code minh họa

```
#include <iostream>
```

```
using namespace std;

void printReverseDigits(int n) {
    if (n == 0) {
        cout << 0;
        return;
    }
    bool first = true;
    while (n > 0) {
        if (!first) cout << ' ';
        cout << n % 10;
        n /= 10;
        first = false;
    }
}

int main() {
    int n;
    cin >> n;
    printReverseDigits(n);
    return 0;
}
```

Bài 8. In các số nguyên tố không vượt quá n

1. Nêu bài toán

Nhập n. Viết hàm void in tất cả số nguyên tố từ 2 đến n.

2. Input

Một số nguyên n, $2 \leq n \leq 10000$.

3. Output

Các số nguyên tố $\leq n$, cách nhau bởi một khoảng trắng.

4. Ví dụ

Input	Output
15	2 3 5 7 11 13

5. Thuật toán giải

Bước 1: Hàm isPrimeLocal(x) có thể được dùng hỗ trợ để kiểm tra x có nguyên tố hay không.

Bước 2: Trong printPrimes(n), duyệt x từ 2 đến n.

Bước 3: Nếu x nguyên tố thì in x.

Bước 4: Quản lý khoảng trắng giữa các số.

6. Code minh họa

```
#include <iostream>
using namespace std;

bool isPrimeLocal(int x) {
    if (x < 2) return false;
    for (int d = 2; d * d <= x; ++d)
        if (x % d == 0) return false;
    return true;
}
```

```

void printPrimes(int n) {
    bool first = true;
    for (int x = 2; x <= n; ++x) {
        if (isPrimeLocal(x)) {
            if (!first) cout << ' ';
            cout << x;
            first = false;
        }
    }
}

int main() {
    int n;
    cin >> n;
    printPrimes(n);
    return 0;
}

```

Bài 9. Vẽ hình chữ nhật bằng ký tự *

1. Nêu bài toán

Nhập số hàng r và số cột c. Viết hàm void in hình chữ nhật r x c bằng ký tự *.

2. Input

Hai số nguyên r, c; $1 \leq r, c \leq 20$.

3. Output

r dòng, mỗi dòng gồm c ký tự *.

4. Ví dụ

Input	Output
3 5	***** ***** *****

5. Thuật toán giải

Bước 1: Hàm drawRectangle(r, c) dùng hai vòng lặp lồng nhau.

Bước 2: Vòng ngoài điều khiển số hàng.

Bước 3: Vòng trong in c dấu *.

Bước 4: Sau mỗi hàng, xuống dòng trừ hàng cuối.

6. Code minh họa

```

#include <iostream>
using namespace std;

void drawRectangle(int r, int c) {
    for (int i = 0; i < r; ++i) {
        for (int j = 0; j < c; ++j) cout << '*';
        if (i + 1 < r) cout << '\n';
    }
}

int main() {
    int r, c;
    cin >> r >> c;
}

```

```

drawRectangle(r, c);
return 0;
}

```

Bài 10. In n số Fibonacci đầu tiên

1. Nêu bài toán

Nhập n. Viết hàm void in n số Fibonacci đầu tiên với $F_0 = 0$, $F_1 = 1$.

2. Input

Một số nguyên n, $1 \leq n \leq 45$.

3. Output

n số Fibonacci đầu tiên, cách nhau bởi một khoảng trắng.

4. Ví dụ

Input	Output
8	0 1 1 2 3 5 8 13

5. Thuật toán giải

Bước 1: Khởi tạo $a = 0$, $b = 1$.

Bước 2: Lặp i từ 0 đến n-1.

Bước 3: In a, sau đó tính $c = a + b$, gán $a = b$ và $b = c$.

Bước 4: Hàm chỉ in kết quả nên có kiểu trả về void.

6. Code minh họa

```

#include <iostream>
using namespace std;

void printFibonacci(int n) {
    long long a = 0, b = 1;
    for (int i = 0; i < n; ++i) {
        if (i > 0) cout << ' ';
        cout << a;
        long long c = a + b;
        a = b;
        b = c;
    }
}

int main() {
    int n;
    cin >> n;
    printFibonacci(n);
    return 0;
}

```

PHẦN II. HÀM SỬ DỤNG THAM SỐ GIÁ TRỊ (THAM SỐ TRỊ)

Tập trung vào cơ chế pass-by-value: tham số nhận bản sao của đối số; thay đổi tham số trong hàm không làm thay đổi biến ở nơi gọi. Các bài ưu tiên hàm trả về một kết quả rõ ràng.

Bài 11. Tính trị tuyệt đối bằng tham số trị

1. Nêu bài toán

Nhập số nguyên x. Viết hàm nhận x theo giá trị và trả về trị tuyệt đối của x.

2. Input

Một số nguyên x.

3. Output

|x|.

4. Ví dụ

Input	Output
-37	37

5. Thuật toán giải

Bước 1: Hàm `absValue` nhận một bản sao của x.

Bước 2: Nếu $x < 0$ thì trả về $-x$; ngược lại trả về x.

Bước 3: main in giá trị trả về của hàm.

6. Code minh họa

```
#include <iostream>
using namespace std;

int absValue(int x) {
    return x < 0 ? -x : x;
}

int main() {
    int x;
    cin >> x;
    cout << absValue(x);
    return 0;
}
```

Bài 12. Tìm số lớn nhất trong ba số

1. Nêu bài toán

Nhập ba số nguyên. Viết hàm nhận ba tham số trị và trả về giá trị lớn nhất.

2. Input

Ba số nguyên a, b, c.

3. Output

Số lớn nhất.

4. Ví dụ

Input	Output
7 21 13	21

5. Thuật toán giải

Bước 1: Gán $m = a$ trong hàm `max3`.

Bước 2: Nếu $b > m$ thì cập nhật $m = b$.

Bước 3: Nếu $c > m$ thì cập nhật $m = c$.

Bước 4: Trả về m .

6. Code minh họa

```
#include <iostream>
using namespace std;

int max3(int a, int b, int c) {
    int m = a;
    if (b > m) m = b;
    if (c > m) m = c;
    return m;
}

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    cout << max3(a, b, c);
    return 0;
}
```

Bài 13. Tính giai thừa n!**1. Nêu bài toán**

Nhập n . Viết hàm nhận n theo giá trị và trả về $n!$.

2. Input

Số nguyên n , $0 \leq n \leq 20$.

3. Output

Giá trị $n!$.

4. Ví dụ

Input	Output
6	720

5. Thuật toán giải

Bước 1: Khởi tạo `result = 1`.

Bước 2: Lặp i từ 2 đến n và nhân `result` với i .

Bước 3: Trả về `result`.

6. Code minh họa

```
#include <iostream>
using namespace std;

long long factorial(int n) {
    long long result = 1;
    for (int i = 2; i <= n; ++i) result *= i;
    return result;
}
```

```
int main() {
    int n;
    cin >> n;
    cout << factorial(n);
    return 0;
}
```

Bài 14. Kiểm tra số nguyên tố

1. Nêu bài toán

Nhập số nguyên n. Viết hàm bool nhận n theo giá trị và cho biết n có phải số nguyên tố hay không.

2. Input

Một số nguyên n, $|n| \leq 10^9$.

3. Output

YES nếu n nguyên tố, ngược lại NO.

4. Ví dụ

Input	Output
29	YES

5. Thuật toán giải

Bước 1: Nếu $n < 2$ thì trả về false.

Bước 2: Duyệt d từ 2 trong khi $d*d \leq n$.

Bước 3: Nếu n chia hết cho d thì trả về false.

Bước 4: Nếu không tìm thấy ước, trả về true.

6. Code minh họa

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; 1LL * d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}

int main() {
    int n;
    cin >> n;
    cout << (isPrime(n) ? "YES" : "NO");
    return 0;
}
```

Bài 15. Tìm ước chung lớn nhất

1. Nêu bài toán

Nhập hai số nguyên dương a, b. Viết hàm dùng thuật toán Euclid để trả về UCLN.

2. Input

Hai số nguyên dương a, b.

3. Output

UCLN(a, b).

4. Ví dụ

Input	Output
84 30	6

5. Thuật toán giải

Bước 1: Trong hàm, khi $b \neq 0$, lưu $a \% b$ vào r .

Bước 2: Gán $a = b$, $b = r$.

Bước 3: Khi $b = 0$, a là UCLN và được trả về.

Bước 4: Các thay đổi a , b trong hàm chỉ tác động lên bản sao tham số.

6. Code minh họa

```
#include <iostream>
using namespace std;

int gcdValue(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    int a, b;
    cin >> a >> b;
    cout << gcdValue(a, b);
    return 0;
}
```

Bài 16. Tính bội chung nhỏ nhất**1. Nêu bài toán**

Nhập hai số nguyên dương a , b . Viết hàm nhận tham số a và b và trả về BCNN.

2. Input

Hai số nguyên dương a , b ; $a, b \leq 10^6$.

3. Output

BCNN(a , b).

4. Ví dụ

Input	Output
12 18	36

5. Thuật toán giải

Bước 1: Tìm $g = \text{UCLN}(a, b)$ bằng hàm `gcdValue`.

Bước 2: Dùng công thức $\text{BCNN} = a / g * b$ để giảm nguy cơ tràn trung gian.

Bước 3: Trả về kết quả kiểu long long.

6. Code minh họa

```
#include <iostream>
using namespace std;
```

```

long long gcdValue(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

long long lcmValue(long long a, long long b) {
    return a / gcdValue(a, b) * b;
}

int main() {
    long long a, b;
    cin >> a >> b;
    cout << lcmValue(a, b);
    return 0;
}

```

Bài 17. Tính lũy thừa nguyên a^n

1. Nêu bài toán

Nhập a và số mũ không âm n . Viết hàm nhận tham số trị và trả về a^n bằng phương pháp nhân lặp.

2. Input

Hai số nguyên a, n ; $-20 \leq a \leq 20$, $0 \leq n \leq 15$.

3. Output

Giá trị a^n .

4. Ví dụ

Input	Output
3 5	243

5. Thuật toán giải

Bước 1: Khởi tạo $result = 1$.

Bước 2: Lặp n lần và nhân $result$ với a .

Bước 3: Trả về $result$.

6. Code minh họa

```

#include <iostream>
using namespace std;

long long powerValue(long long a, int n) {
    long long result = 1;
    for (int i = 0; i < n; ++i) result *= a;
    return result;
}

int main() {
    long long a;
    int n;
    cin >> a >> n;
}

```

```

    cout << powerValue(a, n);
    return 0;
}

```

Bài 18. Đếm số chữ số của số nguyên không âm

1. Nêu bài toán

Nhập n không âm. Viết hàm nhận n theo giá trị và trả về số chữ số thập phân của n .

2. Input

Số nguyên n , $0 \leq n \leq 2 \cdot 10^9$.

3. Output

Số chữ số của n .

4. Ví dụ

Input	Output
105020	6

5. Thuật toán giải

Bước 1: Nếu $n = 0$ thì trả về 1.

Bước 2: Khởi tạo $\text{count} = 0$.

Bước 3: Trong khi $n > 0$, chia n cho 10 và tăng count .

Bước 4: Trả về count ; n gốc ở main không bị thay đổi.

6. Code minh họa

```

#include <iostream>
using namespace std;

int countDigits(int n) {
    if (n == 0) return 1;
    int count = 0;
    while (n > 0) {
        ++count;
        n /= 10;
    }
    return count;
}

int main() {
    int n;
    cin >> n;
    cout << countDigits(n);
    return 0;
}

```

Bài 19. Tính tổng các chữ số

1. Nêu bài toán

Nhập số nguyên không âm n . Viết hàm nhận n theo giá trị và trả về tổng các chữ số.

2. Input

Số nguyên n , $0 \leq n \leq 2 \cdot 10^9$.

3. Output

Tổng các chữ số của n .

4. Ví dụ

Input	Output
50729	23

5. Thuật toán giải

Bước 1: Khởi tạo sum = 0.

Bước 2: Khi $n > 0$, cộng $n \% 10$ vào sum.

Bước 3: Cập nhật $n /= 10$.

Bước 4: Trả về sum.

6. Code minh họa

```
#include <iostream>
using namespace std;

int sumDigits(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

int main() {
    int n;
    cin >> n;
    cout << sumDigits(n);
    return 0;
}
```

Bài 20. Đảo ngược một số nguyên không âm**1. Nêu bài toán**

Nhập số nguyên không âm n . Viết hàm nhận n theo giá trị và trả về số tạo bởi các chữ số của n theo thứ tự đảo ngược.

2. Input

Số nguyên n , $0 \leq n \leq 10^9$.

3. Output

Số đảo ngược của n ; các số 0 ở đầu kết quả không được giữ.

4. Ví dụ

Input	Output
12040	4021

5. Thuật toán giải

Bước 1: Khởi tạo rev = 0.

Bước 2: Mỗi bước lấy digit = $n \% 10$.

Bước 3: Cập nhật $rev = rev * 10 + \text{digit}$ và $n /= 10$.

Bước 4: Trả về rev.

6. Code minh họa

```
#include <iostream>
using namespace std;

long long reverseNumber(int n) {
    long long rev = 0;
    while (n > 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    return rev;
}

int main() {
    int n;
    cin >> n;
    cout << reverseNumber(n);
    return 0;
}
```



PHẦN III. THAM SỐ THAM CHIẾU HOẶC THAM SỐ CON TRỎ (THAM SỐ BIẾN)

Tập trung vào trường hợp hàm cần sửa trực tiếp dữ liệu của hàm gọi hoặc cần trả nhiều kết quả. Một nửa phần dùng reference và phần còn lại minh họa pointer/output pointer.

Bài 21. Hoán đổi hai số bằng tham chiếu

1. Nêu bài toán

Nhập hai số nguyên a và b. Viết hàm nhận hai tham số tham chiếu và hoán đổi trực tiếp giá trị của hai biến.

2. Input

Hai số nguyên a, b.

3. Output

Hai số sau khi hoán đổi.

4. Ví dụ

Input	Output
8 15	15 8

5. Thuật toán giải

Bước 1: Hàm swapRef(int& a, int& b) nhận bí danh của hai biến ở main.

Bước 2: Lưu a vào biến tạm.

Bước 3: Gán a = b và b = temp.

Bước 4: Sau lời gọi hàm, các biến ở main đã thay đổi.

6. Code minh họa

```
#include <iostream>
using namespace std;

void swapRef(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int a, b;
    cin >> a >> b;
    swapRef(a, b);
    cout << a << ' ' << b;
    return 0;
}
```

Bài 22. Sắp xếp ba số tăng dần bằng tham chiếu

1. Nêu bài toán

Nhập ba số nguyên a, b, c. Viết hàm dùng tham số tham chiếu để sắp xếp ba biến theo thứ tự không giảm.

2. Input

Ba số nguyên a, b, c.

3. Output

Ba số sau khi sắp xếp tăng dần.

4. Ví dụ

Input	Output
9 2 6	2 6 9

5. Thuật toán giải

Bước 1: Dùng hàm swapRef để hoán đổi hai biến khi cần.

Bước 2: Nếu $a > b$ thì đổi a, b .

Bước 3: Nếu $a > c$ thì đổi a, c .

Bước 4: Nếu $b > c$ thì đổi b, c .

6. Code minh họa

```
#include <iostream>
using namespace std;

void swapRef(int& a, int& b) {
    int t = a;
    a = b;
    b = t;
}

void sort3(int& a, int& b, int& c) {
    if (a > b) swapRef(a, b);
    if (a > c) swapRef(a, c);
    if (b > c) swapRef(b, c);
}

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    sort3(a, b, c);
    cout << a << ' ' << b << ' ' << c;
    return 0;
}
```

Bài 23. Trả đồng thời giá trị nhỏ nhất và lớn nhất**1. Nêu bài toán**

Nhập ba số nguyên a, b, c . Viết hàm nhận thêm hai tham số tham chiếu mn và mx để trả đồng thời giá trị nhỏ nhất và lớn nhất.

2. Input

Ba số nguyên a, b, c .

3. Output

Giá trị nhỏ nhất và lớn nhất, cách nhau bởi một khoảng trắng.

4. Ví dụ

Input	Output
14 -3 9	-3 14

5. Thuật toán giải

Bước 1: Khởi tạo $mn = mx = a$ trong hàm.

Bước 2: So sánh b với mn , mx và cập nhật.

Bước 3: Làm tương tự với c .

Bước 4: Vì mn và mx là tham chiếu, kết quả được ghi trực tiếp vào biến của main.

6. Code minh họa

```
#include <iostream>
using namespace std;

void minMax3(int a, int b, int c, int& mn, int& mx) {
    mn = mx = a;
    if (b < mn) mn = b;
    if (b > mx) mx = b;
    if (c < mn) mn = c;
    if (c > mx) mx = c;
}

int main() {
    int a, b, c, mn, mx;
    cin >> a >> b >> c;
    minMax3(a, b, c, mn, mx);
    cout << mn << ' ' << mx;
    return 0;
}
```

Bài 24. Trả thương và số dư bằng tham chiếu**1. Nêu bài toán**

Nhập hai số nguyên a, b với b khác 0. Viết hàm tính đồng thời thương nguyên và số dư qua hai tham số tham chiếu.

2. Input

Hai số nguyên a, b ; $b \neq 0$.

3. Output

Thương nguyên q và số dư r .

4. Ví dụ

Input	Output
29 6	4 5

5. Thuật toán giải

Bước 1: Hàm `divMod` nhận a, b theo giá trị và q, r theo tham chiếu.

Bước 2: Gán $q = a / b$.

Bước 3: Gán $r = a \% b$.

Bước 4: In q, r sau lời gọi hàm.

6. Code minh họa

```
#include <iostream>
using namespace std;

void divMod(int a, int b, int& q, int& r) {
    q = a / b;
```

```

    r = a % b;
}

int main() {
    int a, b, q, r;
    cin >> a >> b;
    divMod(a, b, q, r);
    cout << q << ' ' << r;
    return 0;
}

```

Bài 25. Đổi tổng số giây thành giờ, phút, giây

1. Nêu bài toán

Nhập tổng số giây t không âm. Viết hàm dùng ba tham số tham chiếu để tách thành giờ, phút và giây.

2. Input

Một số nguyên t , $0 \leq t \leq 10^9$.

3. Output

Ba số h m s .

4. Ví dụ

Input	Output
7384	2 3 4

5. Thuật toán giải

Bước 1: Tính $h = t / 3600$.

Bước 2: Lấy phần còn lại $t \% 3600$.

Bước 3: Tính $m = \text{rem} / 60$ và $s = \text{rem} \% 60$.

Bước 4: Ghi ba kết quả qua h , m , s là tham chiếu.

6. Code minh họa

```

#include <iostream>
using namespace std;

void splitTime(int t, int& h, int& m, int& s) {
    h = t / 3600;
    int rem = t % 3600;
    m = rem / 60;
    s = rem % 60;
}

int main() {
    int t, h, m, s;
    cin >> t;
    splitTime(t, h, m, s);
    cout << h << ' ' << m << ' ' << s;
    return 0;
}

```

Bài 26. Nhân đôi giá trị bằng tham số con trỏ

1. Nêu bài toán

Nhập số nguyên x. Viết hàm nhận con trỏ tới x và nhân đôi giá trị của biến thông qua phép giải tham chiếu.

2. Input

Một số nguyên x.

3. Output

Giá trị của x sau khi nhân đôi.

4. Ví dụ

Input	Output
17	34

5. Thuật toán giải

Bước 1: Trong main, truyền địa chỉ &x vào hàm.

Bước 2: Trong doubleValue, kiểm tra p khác nullptr.

Bước 3: Thực hiện $*p *= 2$.

Bước 4: In x sau khi hàm kết thúc.

6. Code minh họa

```
#include <iostream>
using namespace std;

void doubleValue(int* p) {
    if (p != nullptr) *p *= 2;
}

int main() {
    int x;
    cin >> x;
    doubleValue(&x);
    cout << x;
    return 0;
}
```

Bài 27. Hoán đổi hai số bằng tham số con trỏ

1. Nêu bài toán

Nhập hai số nguyên. Viết hàm nhận hai địa chỉ và hoán đổi các giá trị tại hai địa chỉ đó.

2. Input

Hai số nguyên a, b.

3. Output

Hai số sau khi hoán đổi.

4. Ví dụ

Input	Output
4 19	19 4

5. Thuật toán giải

Bước 1: Gọi swapPtr(&a, &b).

Bước 2: Trong hàm, đọc *a và *b.

Bước 3: Dùng biến tạm để hoán đổi *a và *b.

Bước 4: Kết quả tác động trực tiếp lên a, b ở main.

6. Code minh họa

```
#include <iostream>
using namespace std;

void swapPtr(int* a, int* b) {
    if (a == nullptr || b == nullptr) return;
    int t = *a;
    *a = *b;
    *b = t;
}

int main() {
    int a, b;
    cin >> a >> b;
    swapPtr(&a, &b);
    cout << a << ' ' << b;
    return 0;
}
```

Bài 28. Tăng mọi phần tử mảng thêm k qua con trỏ

1. Nêu bài toán

Nhập mảng n số nguyên và k. Viết hàm nhận con trỏ tới phần tử đầu mảng, n và k; tăng mọi phần tử lên k.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên. Dòng 3: k.

3. Output

Mảng sau khi mỗi phần tử được cộng k.

4. Ví dụ

Input	Output
5 1 -2 3 0 7 4	5 2 7 4 11

5. Thuật toán giải

Bước 1: Tên mảng a truyền vào hàm được dùng như con trỏ tới phần tử đầu.

Bước 2: Lặp i từ 0 đến n-1.

Bước 3: Thực hiện $*(a + i) += k$.

Bước 4: In mảng sau khi hàm cập nhật.

6. Code minh họa

```
#include <iostream>
using namespace std;

void addToArray(int* a, int n, int k) {
    for (int i = 0; i < n; ++i)
        *(a + i) += k;
}
```

```

int main() {
    int n, a[1000], k;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> k;
    addToArray(a, n, k);
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```

Bài 29. Tìm min và max của mảng qua con trỏ kết quả

1. Nêu bài toán

Cho mảng n số nguyên. Viết hàm nhận mảng bằng con trỏ và trả hai kết quả min, max thông qua hai con trỏ đầu ra.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

Giá trị nhỏ nhất và lớn nhất của mảng.

4. Ví dụ

Input	Output
6 8 -1 12 5 0 7	-1 12

5. Thuật toán giải

Bước 1: Gán $*mn = *mx = a[0]$.

Bước 2: Duyệt các phần tử còn lại.

Bước 3: Nếu $a[i]$ nhỏ hơn $*mn$ thì cập nhật $*mn$.

Bước 4: Nếu $a[i]$ lớn hơn $*mx$ thì cập nhật $*mx$.

6. Code minh họa

```

#include <iostream>
using namespace std;

void arrayMinMax(const int* a, int n, int* mn, int* mx) {
    if (a == nullptr || mn == nullptr || mx == nullptr || n <= 0)
        return;
    *mn = *mx = a[0];
    for (int i = 1; i < n; ++i) {
        if (a[i] < *mn) *mn = a[i];
        if (a[i] > *mx) *mx = a[i];
    }
}

int main() {
    int n, a[1000], mn, mx;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
}

```

```

    arrayMinMax(a, n, &mn, &mx);
    cout << mn << ' ' << mx;
    return 0;
}

```

Bài 30. Tính chu vi và diện tích hình tròn qua con trỏ kết quả

1. Nêu bài toán

Nhập bán kính r . Viết hàm nhận r và hai con trỏ kết quả để tính chu vi và diện tích hình tròn.

2. Input

Một số thực $r > 0$.

3. Output

Chu vi và diện tích trên một dòng, làm tròn 2 chữ số thập phân. Dùng $PI = 3.141592653589793$.

4. Ví dụ

Input	Output
2	12.57 12.57

5. Thuật toán giải

Bước 1: Hàm circleInfo nhận r theo giá trị, perimeter và area là con trỏ.

Bước 2: Gán $*perimeter = 2 * PI * r$.

Bước 3: Gán $*area = PI * r * r$.

Bước 4: main truyền địa chỉ của hai biến kết quả và in với 2 chữ số thập phân.

6. Code minh họa

```

#include <iostream>
#include <iomanip>
using namespace std;

void circleInfo(double r, double* perimeter, double* area) {
    const double PI = 3.141592653589793;
    if (perimeter != nullptr) *perimeter = 2 * PI * r;
    if (area != nullptr) *area = PI * r * r;
}

int main() {
    double r, p, s;
    cin >> r;
    circleInfo(r, &p, &s);
    cout << fixed << setprecision(2) << p << ' ' << s;
    return 0;
}

```

TỔNG KẾT VÀ TÀI LIỆU THAM KHẢO

Bộ bài tập đi từ việc xác định trách nhiệm của một hàm void, đến cách thiết kế hàm trả về kết quả với tham số trị, rồi sang các tình huống cần thay đổi đối tượng bên ngoài bằng reference hoặc pointer. Trọng tâm là nhìn thấy rõ sự khác nhau về luồng dữ liệu giữa ba cách thiết kế hàm.

Gợi ý tổ chức thực hành

- Bài 1-5 - khai báo, định nghĩa, lời gọi và kiểu trả về void.
- Bài 6-10 - kết hợp hàm void với vòng lặp, điều kiện và bài toán số học.
- Bài 11-15 - tham số trị và hàm trả về int/bool/long long.
- Bài 16-20 - hàm tham số trị có hàm hỗ trợ, xử lý chữ số và số học.
- Bài 21-25 - reference parameter, sửa biến gọi và trả nhiều kết quả.
- Bài 26-30 - pointer parameter, mảng qua con trỏ và output pointer.

Điểm cần nhấn mạnh khi giảng dạy

- Hàm void không có giá trị trả về để dùng trong biểu thức; return; nếu xuất hiện chỉ dùng để kết thúc sớm hàm void.
- Với pass-by-value, tham số là bản sao; thay đổi tham số cục bộ không đổi đối số ở nơi gọi.
- Với T&, tên tham số trở thành một cách gọi khác của cùng đối tượng nên phép gán qua tham số có thể thay đổi biến gọi.
- Với T*, bản thân con trỏ vẫn được truyền theo giá trị, nhưng *p cho phép hàm truy cập/sửa đối tượng tại địa chỉ được truyền vào.
- Nếu chỉ đọc qua con trỏ nên cân nhắc const T*; nếu con trỏ có thể không trỏ tới đối tượng thì dùng nullptr và kiểm tra trước khi dereference.

Tài liệu tham khảo chính

Bjarne Stroustrup. The C++ Programming Language, Fourth Edition. Addison-Wesley, 2013. Trọng tâm tham khảo: Chapter 12 - Functions, đặc biệt §12.1 (function declarations/definitions), §12.2 (Argument Passing), §12.2.1 (Reference Arguments), §12.2.2 (Array Arguments); kết hợp §7.2 và §7.7 cho pointers và references.

Ghi chú: bộ bài này dùng các kiểu dữ liệu và bài toán ngắn để làm rõ cơ chế hàm. Trong chương trình lớn, nên ưu tiên giao diện hàm rõ ràng; nếu chỉ cần một kết quả, trả về bằng return thường dễ đọc hơn việc sửa tham số đầu ra.